

Architettura degli elaboratori (6+3 CFU→scritto, laboratorio e orale)

lunedì e mercoledì Spata, venerdì Verga. nove cfu

3 prove; scritto (15 domande a risposte multiple), laboratorio (forse su carta) e orale

Inventore→Federico Faggin, capo progetto e progettista del microprocessore Intel 4004

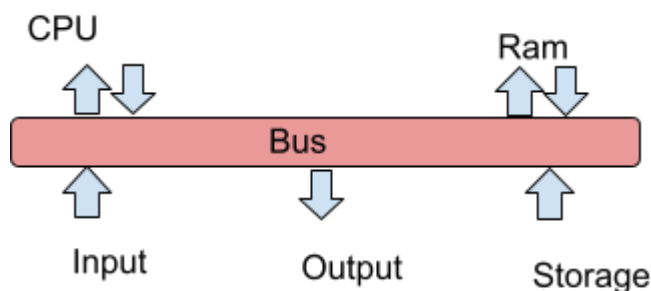
Visionario (porta innovazione, applica le invenzioni)→Steve Jobs, immagina e vede nel futuro, non ha inventato nulla:

- ha introdotto dalla Xerox il sistema a finestre
- ha unito l'input con l'output (l'iPhone e il touchscreen)
- applica le invenzioni
- fonda la Pixar
- aveva un pensiero divergente e creativo (un lateral thinking, non linear thinking)

Altri esempi di invenzioni-innovazioni:

- raggi X di Nikola Tesla, uso in medicina
- reti neurali, Chat GPT

Architettura di Von Neumann (1946)



In rosso il **bottleneck**, collo di bottiglia: il bus. Il bus (= la **scheda madre**) è molto più lento della CPU (anche 1000 volte più lento), genera **inefficienza e latenza** poiché la CPU per accedere ai dati deve **passare** necessariamente attraverso il **bus**. Anche semplicemente per **richiedere** un dato dalla **memoria principale**, la **CPU** deve **passare attraverso il bus**, c'è così una **doppia latenza** per

accedere alla risorsa da **parte** del bus e **della memoria principale**. Il bottleneck avviene perché appunto la velocità del bus è molto minore della velocità della memoria (e soprattutto del processore), tutto si adegua al **worst-case**.

L'esecuzione di un **programma** avviene eseguendo esattamente in **modo ordinato** le istruzioni definite nell'insieme P, ci possono essere salti condizionali e comandi goto (anche se sono sconsigliati poiché aumentano la complessità di lettura).

Genialata (già introdotta da **turing**): la memoria **contiene** sia le **istruzioni**, che i **dati** del programma.

Sebbene la bottleneck principale sia il bus, ci sono tante altre bottleneck fondamentali nell'**implementazione concreta** della **macchina** di von **neumann**:

- la **memoria**, che ha una sua **latenza** (ricordiamo, la cpu ha doppio bottleneck per leggere un dato memoria, il bottleneck del bus e quello della memoria), per questo motivo nelle cpu viene **inserita** la **cache**.
- l'architettura puramente **SISD**, **Single Instruction, Single data**. Per questo nasce **prima** la **pipeline**, e poi l'**architettura superscalare** che include più ALU.
- gli **hazards** nelle cpu

Il Bus

Il bus è il **bottleneck** nella macchina di Von Neumann, **connette** i **moduli** con il **processore**. Processore, **memoria**, e dispositivi I/O si **affacciano** al **bus**, possono leggere e **modificare** i **valori** presenti sulle linee.

Il **system bus** è scomponibile in tre insiemi ordinati di linee: **Control Bus**, **Data Bus** e **Address Bus**.

Segnali di controllo: **I/O-Mem** (0=riguarda l'I/O, 1=riguarda la RAM), **R/W** (Read/Write), **Wait**. Nel bus si utilizza quindi il **concetto master (CPU) slave (RAM, dispositivi I/O)**.

Tutto ciò che passa nel **bus** deve però essere sincronizzato con il **clock del bus** (In Mhz).

Immaginiamo di avere un'architettura a **32 bit** con **32 linee** sia **nell'ABus** che nel **DBus**:

- **ABus**→ 2^{32} , spazio di indirizzamento pari a circa **4 miliardi**
- **DBus**→può trasferire in un'operazione **32 bit**, quindi **4 byte**

Praticamente, in una scheda madre il system bus è realizzato con un chipset (insieme di linee e circuiti) che si occupa di connettere i chip **NorthBridge (CPU-RAM)** e **SouthBridge (CPU-I/O)** tra di loro. Tecnologie usate (in passato) per realizzare il bus:

- ISA e EISA (DBus entrambi con clock 8,33 Mhz, il primo a 8 bit, il secondo 16 bit)
- PCI, Peripheral Component Interconnect, con DBus a 32 bit e clock a 33Mhz
- **SCSI** (Scasi) per i dischi

Oggi anche per le schede grafiche si usa la **PCI Express**: essendo queste ultime molto potenti ad elaborare i dati, il **rallentamento** nei videogiochi è proprio dovuto alla **ridotta bandwidth** del bus.

CPU (processore, Central Processing Unit)

La CPU è un singolo **circuito integrato**, esegue operazioni **logiche, aritmetiche, confronti**, salti **condizionati** e **incondizionati**, è formata da:

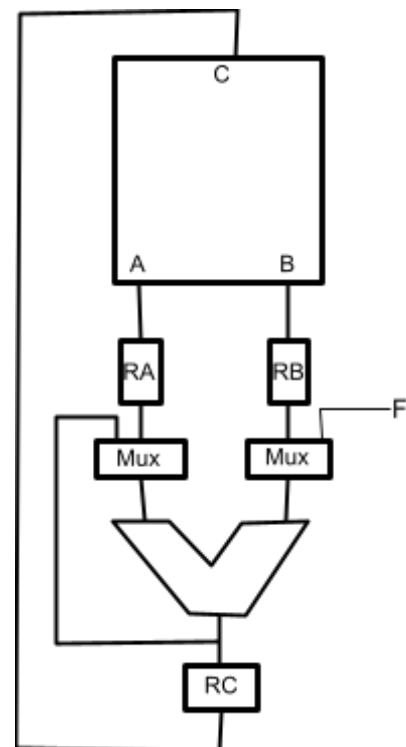
- **instruction register** (istruzione attuale)
- Generatore di indirizzi delle istruzioni e PC (**Program Counter**, prossima istruzione)
- **Banco dei registri** ($R_0 \dots R_{n-1}$) ad **accesso immediato**, è una mini **memoria veloce**,
- la **ALU** (arithmetic, logic unit), che può avere due o 3 operandi (due registri di input e uno di output)
- **Control unit**
- (nelle architetture **CISC** c'è il bus di **interconnessione**)

Banco dei registri e ALU fanno parte del data path, esempio di ciclo di data path:

- gli input vanno dai registri alla **ALU**, la **ALU** fa il **calcolo** e poi imposta il risultato nel **registro di output**.

Il data path è il percorso dei dati all'interno della cpu

- (ricordiamo che le operazioni possono essere fatte **registro-registro** o **registro-memoria**, dove la memoria può essere RAM, cache...)
- notiamo l'importanza del **multiplexer** (aka mux, è uno **switcher**), server perchè la ALU ha solo due linee di ingresso ed una di uscita (la ALU elabora bit), il mux prende due **linee di ingresso** e **produce** un'uscita. Il multiplexer instrada i dati dove servono al momento **giusto**, può instradare valori **immediati**, oppure dal **banco dei registri**, oppure ancora **segnali di controllo**.
- notiamo inoltre che il banco dei registri ha tre porte, **la A e la B sono i due ingressi** per la lettura di due registri, la **porta C è l'uscita** per la scrittura di un registro; per leggere due registri usiamo un singolo



banco di registri con circuiti duplicati oppure due copie del banco dei registri. Il generatore di **indirizzi** delle istruzioni **genera** l'indirizzo della **prossima istruzione** da eseguire del programma e lo **imposta** in **PC** (c'è anche il registro PC Temp, usato per memorizzare il valore di PC in caso di routine)

La CPU è connessa alla scheda **madre** (e quindi la cpu si affaccia al bus→DBUS,ABUS,CBUS) attraverso lo **zoccolo**, potendo così accedere alla RAM per eseguire operazioni di **Load** e **Store**.

La **pipeline** permette alla CPU di funzionare come una **catena di montaggio**: mentre un'istruzione viene **decodificata** (decode), un'altra può essere **eseguita** (execute) e una terza può essere **caricata** dalla **memoria**. Consente di eseguire le istruzioni in parallelo, basta che gli stadi della pipeline non si sovrappongono, in un istante di **tempo T** vengono eseguiti **più stadi**. Anche la **pipeline contribuisce a diminuire il bottleneck del bus**. Per gestire l'esecuzione di più istruzioni nella pipeline si usano i **buffer di interstadio** che memorizzano operandi e operatori, conservando le informazioni tra uno stadio e l'altro. Possiamo introdurre il concetto di **coda delle istruzioni**: essa si forma quando il **tasso di inter-arrivo** (cioè la velocità con cui la CPU elabora le istruzioni) è **minore del tasso di servizio** delle nuove istruzioni. In tal caso, le richieste si accumulano in attesa di essere eseguite. **Issue width**→quante istruzioni il processore superscalare può eseguire contemporaneamente, quante istruzioni lo stage di decode può inserire nello stadio di execute?

Le **operazioni della pipeline** (che ha risorse estremamente limitate) sono regolate dalla Control Unit attraverso l'utilizzo di segnali di controllo e vengono eseguite con una certa frequenza ben precisa (=clock della cpu), ogni fase ha un circuito logico dedicato:

- ottica proattiva→(nelle architetture moderne c'è il **prefetch**, carica gli **opcode** in **cache**, per ridurre i problemi di latenza della **memoria** e aumentare **prestazioni**); può essere eseguito a livello hardware (con una **prefetcher unit** che rileva gli schemi di accesso alla memoria, **pattern**) o via software (lo sviluppatore sa che **cosa** conviene **caricare**).
- **fetch** (Leggo dal **program counter** e carico l'**operation code** passando dalla RAM attraverso il bus); l'op code è un numero che mi identifica un'**istruzione**, associata ad un codice mnemonico; ci sono istruzioni per lavorare tra i registri, ci sono istruzioni per caricare o inserire in memoria dei valori; aumento anche il **program counter di 4** unità per andare alla prossima istruzione
- **decode**, decodifico l'operation code, determino l'**istruzione** e quanti **parametri** mi **servono**, attiva la fase di **operand fetch** (leggo i registri), gli operandi in memoria sono necessariamente accanto all'op code; $RA \leftarrow [R4]$, $RB \leftarrow [R5]$
- **execute** (la control unit dà il comando alla ALU, nelle architetture CISC viene avviato il **microprogramma**, per questo motivo si dice che l'architettura CISC è interpretata) nella fase di execute dobbiamo menzionare la **dispatching unit**, utilizzata in architetture superscalari o con superpipeline, **vedi sotto**.
- **memory** (devo ricevere i dati **dalla Ram** e darli **alla ALU**, questo mi crea **latenza**)
- **write back** (o store, è sottinteso che devo **aspettare, wait, che la ALU finisca**) verso la memoria, o verso i registri IO
- *continuo finchè non ho l'istruzione ALT*

I segnali di **controllo causano overhead**, però sono necessari, noi **vogliamo evitare errori** dentro la cpu per massimizzare le performance.

Controllo Cablato→ I segnali di controllo sono generati dalla control unit attraverso circuiti logici combinatori (componenti fisiche: porte logiche, flip-flop, circuiti combinatori...), non

microprogrammi. È un controllo hardware, un sistema molto veloce, ma meno flessibile (se ci sono bug a livello hardware il prodotto andrebbe ritirato).

L'alternativa al controllo cablato è il **controllo microprogrammato**→i segnali di controllo vengono scritti e i corrispettivi micro programmi vengono scritti in memorie di controllo.

È più flessibile (il comportamento della CPU può essere cambiato **aggiornando** i microprogrammi, quindi il **firmware**), ma è più **lento** e **complesso** da **gestire**.

Esempio di segnali di controllo: **WMFC** (Wait Memory function complete) e **MFC** (Memory function completed) sono usati per la gestione di ritardi nell'accesso alla memoria.

Se una periferica ha richiesto l'**interrupt**, il processore sospende il processo attuale ed esegue l'**Interrupt Service Routine** (l'**interrupt** crea un problema perchè bisogna mantenere l'**integrità** poi nel tornare alla **normale esecuzione**).

I problemi principali della pipeline:

Gli **hazard** possono addirittura dimezzare le performance.

Data Hazard

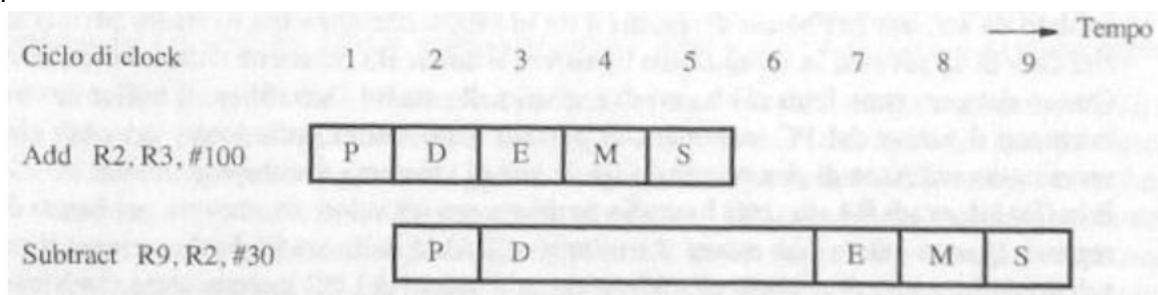
I data hazard indicano nella pipeline dei conflitti causati da istruzioni i cui risultati dipendono dalle istruzioni precedenti, quindi modificando l'ordine non si rispetta la semantica del programma.

Dipendenza **RAW**:

Esempio $x = a + b$; $y = x + c$; queste non possono essere parallelizzate (tranne se ricorriamo alla sincronia), devono essere eseguite in ordine. I data hazard possono causare **stallo** nella **pipeline**, in rarissimi casi il **roll back**.

L'esempio tipico di data hazard è la dipendenza **RAW**, **Read after Write** (

$R_1 = R_2 + R_3$; $R_4 = R_1 + R_5$), che rappresenta una dipendenza **vera**, o di **flusso**, è praticamente l'unica che crea un vincolo reale.



Dipendenze che non creano grandi problemi:

- **WAR**→Write After Read ($R_1 = R_2 + R_3$; $R_2 = R_4 + R_5$)
- **WAW**→Write after Write, $R_1 = R_2 + R_3$; $R_1 = R_4 + R_5$

Le dipendenze **WAR** e **WAW** vengono risolte **senza stalli** (=senza l'uso di **nop**) dall'algoritmo di Tomasulo che utilizza il register renaming. Esempio di Write after Write:

- $h = a + b$
- $h = c + d$

Queste due istruzioni in teoria userebbero lo stesso registro e quindi dovrebbero essere eseguite in modo sincrono, per evitare ciò rinominiamo il secondo registro h in t1.

Con il **register renaming** quindi, i processori hanno più **registri fisici** rispetto a quelli presentati nell'ISA. Per esempio, 32 registri architetturali, e 100 fisici; viene così eseguito il mapping dinamico con una Register Alias Table.

structural hazard (risorse limitate)→contese per l'accesso ad una stessa risorsa (**thread safety**), in questo caso si usano istruzioni **NOP**, No Operation, istruzioni **fittizie** che fanno eseguire **lavoro inutile** al core, tipo fase wait. Oppure la pipeline va in stallo
Esempio: due **istruzioni** che cercano di **accedere contemporaneamente** alla **memoria**
Metafora→il problema dei 4 filosofi a cena con un'unica forchetta.
Una possibile soluzione è considerare **due cache**, quella delle **istruzioni** e quella dei **dati**.

control hazard (=ritardi nei salti)→quando abbiamo istruzioni di **salto** che **alterano** il **flusso** del **programma** (il **program counter**). Avviene con le istruzioni branch(if) e **goto**.
Vedi speculative execution sotto.

Cache miss (=ritardi nell'accesso alla memoria)→anche perdite fino a 10 cicli di clock. Spesso il dato in un determinato ciclo di clock non è available, senza i dati necessari se provassimo a continuare avremmo una situazione di **hazard** con **possibile stallo**.
Si usano i segnali WMFC e MFC, in questo caso gli stadi critici nell'accesso alla memoria della pipeline sono **memory** e **write back**.
La soluzione più semplice prevede l'utilizzo delle **NOP**, le operazioni bolla.
L'altra possibile soluzione è il **prefetching**, precaricamento, nel caso l'out of order execution consente di evitare lo **svuotamento** della **pipeline**.

Tecniche per ridurre la dipendenza dai dati (e aumentare le performance)

Per ridurre gli effetti dei **data hazards** e dei **cache miss** (=lo stallo), in generale quando il dato è unavailable si utilizza la tecnica dell'**out of order execution**, il cui obiettivo è prevenire lo svuotamento della pipeline (che avviene quando un'istruzione deve attendere il completamento di un'altra da cui dipende, ciò genererebbe una condizione di stallo della pipeline perchè deve aspettare che l'istruzione precedente venga completata) mantenendola attiva. Se abbiamo due istruzioni dipendenti come $x=a+b$ e $y=x+c$ e due istruzioni indipendenti come $f=t$ e $p=m$, posso eseguire contemporaneamente nella pipeline le istruzioni indipendenti (come $x=a+b$, $f=t$, $p=m$), salto temporaneamente l'istruzione dipendente $y=x+c$ che eseguirò appena l'istruzione da cui dipendeva è conclusa.
I passaggi dell'out of order execution sono quindi:

- individuare le istruzioni **dipendenti** e **indipendenti** (l'analisi delle etichette può essere fatta dalla **CPU** stessa, oppure dal **compilatore**, le Very Long Instruction Word). La raccolta delle **istruzioni** è il lavoro della **dispatch unit**.
- selezionare le istruzioni future indipendenti (questo viene fatto usando proprio le reservation stations, che indicano quando un'istruzione può essere eseguita)
reservation stations: buffer che conservano istruzioni, operandi e operatori oppure la raccolta e etichettatura delle istruzioni indipendenti è fatta direttamente dal compilatore (architetture basate sulle **Very Long Instruction Word**)
- eseguire quelle **istruzioni** in **anticipo** (anche se nella **sequenza** del **programma** andrebbero **eseguite** dopo, e potrebbero essere anche **buttate**), e poi man mano si sbloccano le **istruzioni** che prima **avevano** la **dipendenza** che ora è stata **risolta**. Questo è il lavoro della commitment unit (=reorder buffer, buffer velocissimo in SRAM), le istruzioni nelle reservation stations vengono inserite secondo la coda di questo buffer.

La pipeline non viene così interrotta, rispetto all'esecuzione in ordine (dove viene usata al 40%), otteniamo anche un **85%**, quindi l'utilizzo della **pipeline** è **massimizzato**, 2-3x

performance **improvement**. È una tecnica apparentemente **semplice**, però sofisticata: il processore deve infatti **continuamente controllare** che non venga alterata la **semantica** del programma e deve essere in grado di **ricostruire** l'ordine del **programma** non appena finisce di gestire le **istruzioni con dipendenze** (**alterna in order e out of order execution**). Uno dei problemi più critici è quando viene interrotto nel bel mezzo della fase di out of order execution, a causa di un **interrupt**. In questo caso il processore deve essere in grado di tornare in modalità in-order execution e di rimettere le istruzioni al loro posto (il **precise exception handling**).

La speculative execution

La pipeline quando ci sono salti (come i comandi **goto/branch-if**) deve capire **dove andare** (altrimenti sarebbe costretta a fermarsi), e fa una **branch prediction** (il branch sarà taken o no? E se sì, dove mi porterà?).

Perché deve capire **dove andare**? La predizione è importante per mantenere la **pipeline piena**, così non si sprecano **cicli di clock**, possiamo eseguire l'analisi delle dipendenze per l'Out of order execution correttamente e non lasciamo stadi in attesa.

Se ci sono problemi, o sbaglia la previsione, fa **ripartire la pipeline** (il **rollback**, con i **control hazard** è costretta a ripristinare l'ultimo stato corretto). Le previsioni sono:

- predizione statica, si basa su semplici regole e considera il tipo di salto, la posizione dell'istruzione... (non la usa nessuno)
- **Dynamic Branch Prediction** (si adatta in base al comportamento del programma), precisissima, si basa **sullo storico**, è una predizione **statistica, probabilistica, non deterministica** (viene eseguita **in parallelo dalla branch prediction unit**). La BPU ricerca dei **Pattern** (schemi che si ripetono nel tempo con un certo periodo), così facendo otteniamo una precisione in media del **90-95%**.

La Branch Prediction Unit mantiene un **Branch Target Buffer** (=buffer di destinazione di salto), una sorta di cache che **contiene** gli **indirizzi** di istruzione del salto, l'**indirizzo** di destinazione del salto, e i bit per l'algoritmo di prediction e una **Pattern History Table**, che appunto contiene lo storico (associa al branch un valore tra Mostly Taken, Taken, Not Taken, Mostly Not Taken).

PHT→Il salto verrà fatto?

BTB→Se sì, dove andrò dopo?

Nella gestione dei salti, si possono adottare due approcci:

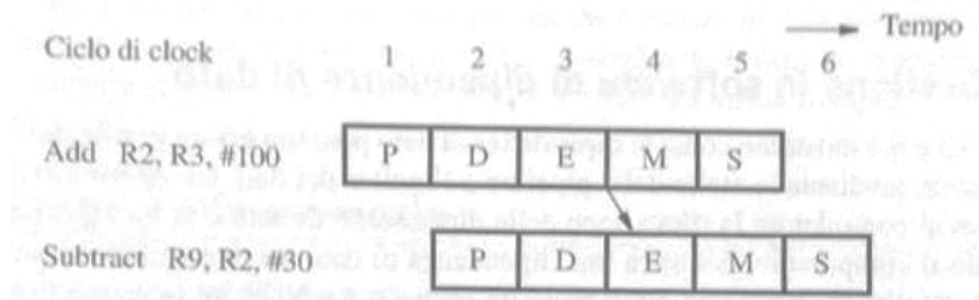
- **normale**, le istruzioni dopo il **salto sono scartate**
- **differito**, le istruzioni dopo il salto **vengono conservate ed eseguite in ogni caso** (il compilatore proverà a organizzare le istruzioni in modo da mettere dopo un salto istruzioni che vanno comunque eseguite)

Il salto differito è un approccio conveniente usato nelle **vecchie architetture RISC**, se il branch predictor dovesse fallire, molto probabilmente non c'è bisogno di fare il rollback della pipeline (perché la pipeline è riempita con altre istruzioni), ma si può **continuare l'esecuzione**.

Purtroppo la speculative execution è difficile da implementare, per questo motivo è essa stessa una **potenziale vulnerabilità** (vedi i casi di **Meltdown** e **Spectre**).

Il prefetch (vedi parte sopra, operazioni della pipeline)

Forwarding→inoltre degli operandi, trasmette da un'istruzione produttrice ad una consumatrice direttamente, saltando il passaggio memory (=il salvataggio nei registri)

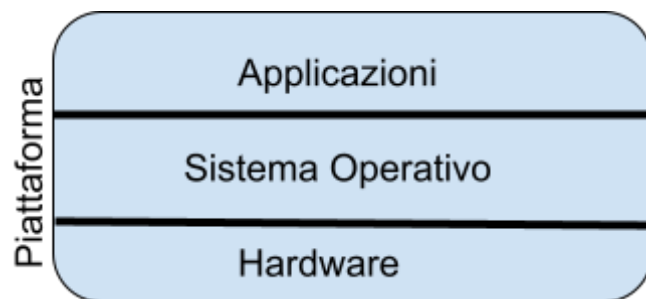


NOP, No Operation → operazioni bolla, la cpu è messa in condizione di stallo.

Le performance del processore dipendono quindi dalla **frequenza di clock**, dall'**efficienza dell'architettura**, dal numero di cicli **richiesti** per ogni istruzione, le **velocità della cache**... In realtà nelle CPU moderne non ci sono solo la ALU, i registri e la control unit, ma anche la Floating Point Unit, unità specifica per il calcolo vettoriale SIMD, e altre unità specializzate in calcoli complessi. **Stack dell'informatico**

Cache

Le CPU includendo **dato** ricevuto spostato nelle essere **nesso**



vengono realizzate anche la **Cache**: il dalla **ram** viene **varie cache**, per poi nel **registro**. La

cache è molto più vicina alla **CPU** rispetto alla **ram**, inoltre non deve passare per il **bus generale**, ma ha un suo bus apposta (il **bus di cache**).

I tipi di cache sono:

- **L1** è la più veloce, **32-64 Kb**; **1-2** cicli di clock per accedervi; **attenzione**: non è che siccome è la più veloce mettiamo tutti i dati qua, è sconsigliato perché è piccola e si riempie subito)
- **L2**, **256kb-1Mb** **5-10** cicli di clock per accedervi
- **L3** è lenta, **8-32 Mb**, **20-40** cicli (per la ram ci vogliono centinaia di cicli), però è condivisa con tutti i core (qui si presenta il problema del race condition → **MESI**)

Approccio / Principio di località temporalità:

LRU → **Least Recently Used** → il dato viene usato poco? lo sostituisco

La cache serve ad alleggerire il bus, infatti è realizzata con tecnologia **SRAM**.

Cache hit/miss:

- il processore, prima di leggere un dato in **ram**, vede se c'è **nella cache** (hit) passando attraverso il **bus di cache** (sì, c'è un bus, però è interno alla cpu e riservato solo alla cpu)
- se non c'è (miss) mette in cache dalla RAM il **dato**, e i **dati ad esso adiacenti** (pensiamo agli array, che sono tutti vicini)
- nel caso migliore in 4 cicli la CPU legge dalla cache L1
- nel caso peggiore legge dalle tre cache **L1, L2, L3** e poi dalla **RAM** (più di 300 cicli)

Per evitare structural hazard usiamo il protocollo **MESI** (Modified, Exclusive, Shared, Invalid) che serve a sincronizzare i dati in memoria RAM con i dati in memoria cache, per evitare letture incoerenti. Shared-Exclusive → uguale a quello in RAM (S=in più cache; E=solo qua)

Architetture superscalari

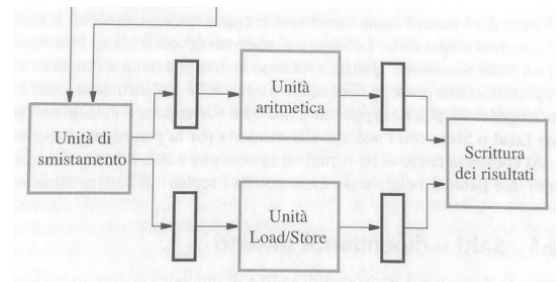
Nella pipeline lo stadio più lento è l'**execute**, c'è una sola **ALU** che svolge un'operazione alla volta richiedendo un determinato tempo di esecuzione (quindi al massimo **1 istruzione per clock**).

Per accelerare aggiungiamo più unità di esecuzione (più **ALU**, più **BPU**, più **LoadStoreUnit**, più **FPU**), ottenendo processori superscalari, più rapidi, aumentiamo il **throughput**.

Le architetture **superscalari** eseguono quindi più **istruzioni per clock** e possono eseguire parallelamente più istruzioni **indipendenti**.

C'è quindi la **dispatching unit**, l'unità di smistamento, che **distribuisce** le **istruzioni** dopo la fase di **decode** alle **unità apposite**. È una sorta di scheduler:

- verifica che le istruzioni siano **state caricate** e che i registri delle unità **apposite siano disponibili**
- tutto ciò **per prevenire il deadlock** (una **doppia dipendenza**, due istruzioni che rimangono entrambe bloccate perchè vi sono dipendenze reciproche).
- per garantire l'**ordine di esecuzione** usiamo le **reservation stations**



A parità di istruzioni di un programma, più **aumentano i core**, più aumenta la **probabilità di avere un deadlock**.

per fare questo ogni unità di esecuzione contiene le **reservation stations**, buffer che gestiscono le dipendenze tra i dati conservando istruzioni, operandi e operatori, e bloccando l'esecuzione di un'istruzione finchè tutti i suoi operandi sono **disponibili** (reservation stations=il **parcheggio delle istruzioni, tu aspetti qua**).

Le prime architetture superscalari furono **RISC** (gli Sparc di **Sun Microsystems** per esempio), e nacquero **proprio** per i **server**.

Primi processori Intel ad architettura superscalare: il **Pentium D** e il **Core Duo**.

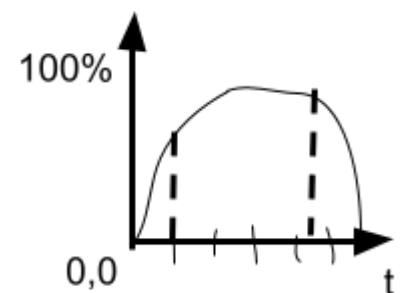
PS: Se un processore è a single core, l'apparente multi-tasking che possiamo osservare è in realtà virtuale, un parallelismo virtuale gestito dal sistema operativo.

Problema dello scheduling e della previsione dei tempi

Nei processori con **architetture superscalari** (quelli **odierni**), in particolare la cpu deve capire per quanto tempo verrà usata la **memoria**, che **rappresenta** in un **elaboratore** la risorsa limitata principale.

Il concetto alla base è lo **stesso** del problema dei 4 filosofi, però nel caso delle CPU è molto più complesso.

La cpu deve **prevedere** quanti cicli di **tempo** richiedono le istruzioni, per poter ottimizzare l'accesso alle risorse: si crea **infatti** una condizione di **race condition** (gara per l'accesso alle risorse, che sono limitate e condivise).



Architettura e Implementazione

L'Architettura: ciò che è visibile al programmatore: l'**ISA**, i modelli di **memoria**, i registri, le modalità di Input/Output. L'implementazione invece indica come l'architettura viene implementata fisicamente (il layout dei chip, l'elettronica, il circuito, l'ottimizzazione interna).

Un ISA (**Instruction Set Architecture**) può essere anche **implementato**, non solo a livello **hardware**, ma anche a **livello software**: parliamo di **interpreti** ed **emulazione**.

Un'istruzione ISA ha quindi:

- Un **operation code**
- Una **lunghezza** in **byte** includendo gli operandi
- Un preciso numero di **cicli** di **bus** per il completamento

RISC	CISC
Set di istruzioni complesso (fase di decode molto lunga). Il datapath è molto più lungo e perchè c'è il bus di interconnessione, si può creare congestione.	Set di istruzioni ridotto, datapath con un singolo passo (una word per istruzione).
Nato da colossi come IBM e Intel	Nato per i server, ricordiamo gli Spark di Sun microsystems (Ultra sparc e SuperSparc).
Tante istruzioni già pronte da usare, facile da programmare in assembly.	I programmi sono più lunghi, ad una istruzione CISC corrispondono più istruzioni RISC
Molto energivoro	Energy friendly
Portabilità (si tende a mantenere anche le istruzioni legacy, vedasi x86)	In costante aggiornamento

L'ISA (Instruction Set Architecture) nei processori moderni è un ibrido tra **CISC** (complex) e RISC (reduced), il **CRISC**. Otteniamo i vantaggi di entrambe le architetture, sia la portabilità del CISC, che l'efficacia del **RISC**. Internamente sono basati su principi **RISC** (quindi le operazioni **semplici** e **comuni** vengono eseguite direttamente in modalità **RISC**, le operazioni più **complesse** o **legacy**, sono eseguite utilizzando **micro programmi** che vengono convertiti in **RISC**, questo potrebbe apparentemente sembrare una **bottleneck**, ma in realtà rispetto al **bottleneck** del **bus** e della **memoria** è **perfettamente trascurabile**).

Il problema della sicurezza

Bisogna proteggere i dati sensibili (password, bancari, biometrici) a livello hardware anche se il software è compromesso. Si usano i TEE, **Trusted Execution Environment**:

- Intel **SGX** (che racchiude i software in sandbox)
- ARM **TrustZone** ()
- AMD **Memory Guard** → cifra la memoria

Architetture riconfigurabili

I FPGA (Field Programmable Gate Arrays) sono porte logiche a livello **hardware programmabili** (alcune programmabili **one time**, la maggior parte anche a **posteriori**, a volte anche in tempo **reale dinamicamente**). Riducono i consumi **energetici**, aumentano l'**efficienza**. Alcuni esempi di utilizzo sono dispositivi che implementano algoritmi di **hashing**, o di **crittografia**, e dispositivi di **encoding** e **decoding** di video.

Memoria

Memoria **primaria-centrale** (in ordine di velocità)→**registri, cache (L1,L2,L3), RAM**

Memoria secondaria-di-massa (più lenta)→**ssd, hdd, EEPROM** (per la fase di bootstrap)

Tecnologie per le memorie (obiettivo→**ridurre la latenza costruttiva**):

- **DRAM**→**Dynamic**, usano **micro condensatori**
- **SRAM**→**Static**, usano circuiti **flip flop** (quella delle cache)
- **3D stacking**→per ottimizzare la net si usa la 3° dimensione, **High bandwidth memory**

La **RAM** (installata negli slot DIMM) è una memoria **volatile**, è un vettore di **word** ben ordinato che contiene istruzioni del **software** e dati di **Input** e **Output**.

word→dimensione dei dati che un **processore** può elaborare in un **ciclo di clock**:

- 8 bit - 16bit→microcontrollori e i **primi processori** (1-2 byte)
- 32 bit→microcontrollori moderni (come le **board nucleo** di ST) e vecchi processori
- 64 bit→lo standard per le architetture moderne (=8 byte)

Aumentando la dimensione della **word** aumenta il limite **massimo** per gli interi che posso rappresentare, però diminuisce la **performance** e aumenta la **complessità** del **circuito**.

L'insieme di tutte le word di una memoria (ognuna associata ad un indirizzo) è detto **spazio**

di indirizzamento, la prima word ha indirizzo **0**, l'ultima $2^n - 1$.

L'ampiezza dello spazio di indirizzamento è determinata dall'ampiezza del bus di indirizzamento.

Un singolo processo in architettura 32 bit può leggere fino a 2^{32} celle, 4Gb.

Questo stesso limite dei 4Gb per file si presenta anche nelle chiavette, quando sono formattate in Fat32.

Dispositivi di Input/Output

I dispositivi I/O consentono all'utente di **interfacciarsi** con l'**elaboratore**.

La comunicazione tra CPU e dispositivi I/O avviene attraverso il **bus**, la periferica si interfaccia al **Control bus**, si sincronizza con il suo clock, e può passare i **dati** alla **CPU** in due **modi**:

- **Interrupting**→il dispositivo, **attivando** delle linee del Control Bus, chiede alla CPU di fermarsi per **elaborare** una **Interrupt Service Routine**.
- **DMA (Direct Memory Access)**→un'area specifica della memoria viene **mappata** ad **hoc** per la **periferica**, il processore attua poi il **polling** (osserva **costantemente** i valori in **memoria** e **verifica** se ci sono **nuovi dati**).

Servono dei circuiti appositi per gestire il passaggio di dati, rispettivamente l'**Interrupt Controller** e il **DMA Controller**.

L'interrupting è **asincrono** (e tutto ciò che è asincrono è parallelizzabile), se però la CPU stava attuando una **out of order execution**, ci sono molte **difficoltà** da **gestire**.

Con la nascita delle interfacce **GUI** (eg. Windows 95) bisogna rendere il computer **user-friendly**: con Windows 95 il sistema operativo prova ad impostare le periferiche autonomamente (tecnologia **Plug and play**), consentendo anche l'**hot swapping**.

Tutto ciò grazie alla nascita di diversi standard, protocolli con regole ben precise: **USB, Ethernet, SATA**. Sono in disuso le seriali PS/2 e le porte parallele.

Storia elaboratori

Dalla macchina analitica di **Babbage** (che usava schede **perforatrici**, 19° secolo) passiamo alle **valvole** (20° secolo) con l'**ENIAC** e poi l'**EDVAC** (basato su architettura di Von Neumann).

Poi nascono i transistor nel **1947** sviluppati da **Bell Labs**. I transistor consentono di realizzare circuiti (i cosiddetti **integrated circuits**) piccoli, **economici** ed **efficienti** (usano il **90%** di energia **in meno** delle valvole).

Il primo processore nacque nel 1971, l'**Intel 4004** con 2300 transistor. Le evoluzioni furono l'**Intel 8080** (8 bit), l'**Intel 8086** (16 bit), l'**Intel 80386** (32 bit).

Legge di Moore→ogni due anni circa nei chip **raddoppia** il numero di **transistor**; c'è però un limite fisico: la **dimensione** e il **calore**.

La risposta ai limiti fisici è data proprio dall'era dei **processori multicore**, le architetture **superscalari** (si tende quindi a **parallelizzare**, preferire l'**asincronia**).

Il primo processore dual core fu il **Pentium D**, AMD debuttò in quest'era con l'**Athlon 64x2**. Man mano aumentano la frequenza di clock ma soprattutto il **numero di core** (oggi nei server abbiamo anche **128** core, nelle cpu **desktop 16**).

Per **parallelizzare ancora** di più vengono introdotte nuove **architetture**, come le **GPU** (1000+ core).

Il calcolo per le AI richiede architetture **ancora** più **specifiche** (operazioni di **convolution** e **moltiplicazione** tra **matrici**, 4 cicli for annidati), per questo ci sono unità come le **TPU** di **Google** e le **NPU**.

La maggior parte delle **architetture** di oggi poi sono **eterogenee** (applichiamo l'**offload computazionale**), se prendiamo come esempio i processori Apple serie M contengono una **CPU**, una **GPU** e un'unità **NPU** (ottimizzate per l'**inferenza**→delta Valore reale - valore predetto). Ma ci sono anche core per la **sicurezza**, **DSP** per segnali audio-video...

Il chip Apple M1 è proprio un SoC (**System on Chip**), nei **SoC** i moduli sono interconnessi tra di loro e comunicano attraverso una net usando dei **protocolli standardizzati**.

Il futuro è nel **computer quantistico** (per la crittografia), nel computing **fotonico** (usando i raggi di luce) e nel **molecular computing**.

Gestione dei consumi

Le cpu, e specialmente le gpu, consumano **molta energia**. C'è quindi il problema di ridurre il calore da dissipare, l'energia da **usare**, e salvaguardare l'**eventuale batteria**.

Tecniche utilizzate:

- **DVFS**→**Dynamic Voltage and Frequency Scaling** (adattare dinamicamente clock e voltaggio in base al carico)
- **near threshold computing**→si utilizza il threshold minimo nei transistor
- **dark silicon management**→alcune aree del chip vengono spente per evitare il sovraccarico

Addizionatori bit a bit

Ogni operazione **aritmetica** può essere **tradotta** in **equazioni booleane**, quindi in circuiti logici creati utilizzando delle **porte logiche**.

Il circuito per l'**operatore somma** è composto da diverse parti, si può **ricostruire** pensando a come **eseguiamo** l'addizione in colonna **bit a bit**.

Serve un circuito per sommare **due bit** (ed eventualmente il **riporto in ingresso**): due **half adder** insieme!

Un half adder prende in **ingresso** i **due bit** e **restituisce** la **somma** (=xor) e se c'è **riporto** (=and).

Spesso però potremmo avere un riporto proveniente dalla **precedente somma** bit-bit. Per questo serve un altro **half adder**.

Questo secondo half adder ha come input la **somma** del primo **half adder** e il **riporto** in **ingresso**, l'output è la **somma definitiva** bit-bit. Il riporto definitivo corrisponde all'**OR** tra i **due riporti** dei due half adder.

Il Full adder ha di conseguenza **tre ingressi** e **due uscite**, i due bit da sommare del nostro numero, il riporto e la somma e il riporto da propagare. Nella somma per verificare se c'è un **overflow**, un trabocco, ovvero quando i bit della nostra **architettura non sono sufficienti** per **rappresentare** il **numero**, basta verificare il riporto dell'ultimo **full adder**, quindi del **bit più significativo**, se è uno abbiamo un overflow.

Il sistema proposto propaga **sequenzialmente** i **riporti** e questo ovviamente **crea un ritardo**, per questo motivo come alternativa al **ripple carry adder** si utilizza il **carry lookahead adder**.

Caratteristica	Ripple Carry Adder (RCA)	Carry Lookahead Adder (CLA)
Velocità	Lenta, $O(n)$	Alta $O(\log n)$, calcolo dei riporti parallelo
Complessità Hardware	Bassa (poche porte logiche, ripple propagato)	Alta (molte porte logiche) Servono funzioni di generazione (prodotto logico) e propagazione (somma logica)
Area Occupata	Minima	Significativa
Costo	Economico	Elevato (anche per i consumi energetici)

Sottrazioni con addizionatore

Il nostro addizionatore **in effetti** è capace di eseguire le **sottrazioni**, dobbiamo solo adottare qualche piccola accorgimento. Ricordiamo che per fare le sottrazioni in binario il sottraendo lo convertiamo utilizzando proprio il **complemento a 2** che consiste nel complemento a uno (quindi **invertire i numeri**) e poi nel **sommare il numero 1**.

Applichiamo questa procedura per convertire il secondo numero in complemento a due.

La parte più semplice è **sommare uno**, ciò viene fatto impostando il **riporto di ingresso** nel bit meno **significativo** ad uno (di default è **zero**).

Per invertire i bit, mettiamo delle porte **xor davanti** prima degli half adder negli **input** che **corrispondono** ai bit del **nostro numero**.

Abbiamo creato una mini alu, la nostra mini alu però non sa quando eseguire l'operatore somma e quando l'operatore differenza, inseriamo un **bit otype** che indica proprio che operazione bisogna fare: **0=somma**, **1=sottrazione**. Il bit otype si comporta come un segnale di controllo.

Il riporto in ingresso del bit meno significativo viene posto al **valore di otype** (così se è uno sommiamo uno al secondo elemento).

Il bit otype viene anche usato nelle porte logiche **xor** usate per **invertire** il secondo numero (trasformarlo in **complemento a uno**): se è 0 non succede niente, se è 1 avviene l'inversione.

Trabocco per somma: Basta verificare se il riporto del bit più **significativo** è **uno**.

Trabocco per sottrazione: Basta verificare se $\text{xor}(\text{riporto_uscita}, \text{riporto_ingresso})$ è uno.

Il nostro addizionatore dunque, con un **solo circuito**, fa sia **somme** che **sottrazioni**.